

Experiment 2

Module Name: Automation with Selenium

Experiment Title: Synchronization & Explicit Waits

Problem Statement: Interact with dynamic web content that takes time to load without using hardcoded time.sleep().

Objective: To implement WebDriverWait and expected_conditions to handle asynchronous element rendering.

Tools, software, and concepts used: Python, Selenium, WebDriverWait, expected_conditions (EC), By.CSS_SELECTOR.

Source code or implementation steps:

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

driver = webdriver.Chrome()
driver.maximize_window()

try:
    driver.get("https://the-internet.herokuapp.com/dynamic_loading/2")

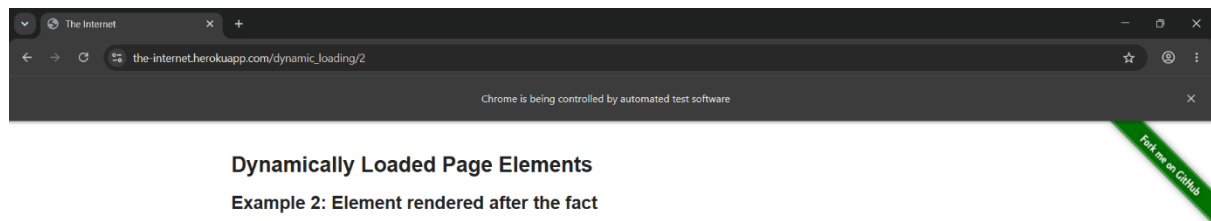
    # Click Start
    driver.find_element(By.CSS_SELECTOR, "#start > button").click()

    # Explicit Wait for the loading text to appear (Max 10 seconds)
    wait = WebDriverWait(driver, 10)
    hello_world_element = wait.until(
        EC.visibility_of_element_located((By.ID, "finish"))
    )

    result_text = hello_world_element.text
    assert "Hello World!" in result_text
    print(f"Test Passed: Extracted text is '{result_text}'")

finally:
    driver.quit()
```

Output with clearly labelled screenshots:



Screenshot 1: Loading bar visible on screen



Screenshot 2: "Hello World!" text displayed

- **Console Output:** Test Passed: Extracted text is 'Hello World!'

Brief Result: Script successfully waited for the hidden element to render before extracting text.

Observation: Explicit waits halt script execution *only* as long as necessary, unlike `time.sleep()`, making the test faster and less flaky.

Conclusion: `WebDriverWait` is essential for handling dynamic/AJAX content reliably.